



Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura

Cátedra: Paradigmas y Lenguajes

Año:2026

Carrera: Licenciatura en Sistemas de Información

Grupo N.º 13

Integrantes:

Abraham Fernández — GitHub: abrahamfernand-pixel

Camila Paloma Hornos — GitHub: CamilaHornoss

Natalia Magali Lezcano — GitHub: lezcanomagali32-commits

Valentín Nicolás Luque — GitHub: Valenluq10

Verónica Stefanía Gómez Varela — GitHub: VeroSGV

Lenguaje Principal: Common Lisp

Lenguaje Comparativo: Scala

Librería externa elegida: Local-time

Índice general

1.INTRODUCCION.....	3
2.DESARROLLO.....	4
2.1- Fundamentos del diseño funcional adaptado	4
2.1.1- Definición del paradigma Funcional.....	4
2.1.2- Principios Fundamentales de la Programación funcional.....	4
2.1.3- Principios Aplicados al Sistema.....	5
2.1.4- Implementación del sistema de Semáforos	5
2.2.1-Justificacion de Librerías.....	9
2.2.2-Quicklisp y Librerías Seleccionadas	9
2.2.3-Eleccion de Librería Local-Time.....	9
2.3.1-Bitacora de Errores y Depuración.....	9
2.3.2- Función Transición.....	10
2.3.3- Función Timer.....	11
2.3.4- Función Log-auditoria.....	12
2.3.5-Funcion Duración-Ciclo.....	13
2.3.6-Funcion Suma-Tiempo.....	14
2.4.1-Parte Teorica Comparativa.....	16
2.4.2- Introducción al Lenguaje Scala.....	16
2.4.3-Comparacion de funciones en Lisp y Scala.....	19
2.4.4-Estructura del Semáforo en Scala	22
2.4.5-Comparacion entre listas en Scala y Common Lisp.....	23
3-Conclusion.....	24
4-Referencia Bibliográfica.....	25

Introducción:

En el presente informe queremos dar a conocer el desarrollo del proyecto de semáforos inteligentes llevado a cabo en la asignatura Paradigmas y Lenguajes de Programación. A través de este trabajo tuvimos la oportunidad de aplicar y poner en práctica los conceptos fundamentales del paradigma funcional mediante la resolución de una problemática del mundo real, integrando los conocimientos adquiridos a lo largo de la cursada.

A lo largo de este informe, se brindará información sobre el desarrollo e implementación de las distintas funciones requeridas para el sistema de semáforos, explicando su funcionamiento y las herramientas utilizadas para su construcción. Asimismo, se presentará la incorporación del gestor de paquetes Quicklisp, explicando su propósito, funcionamiento y utilidad, dejando ver las diferencias observadas del antes y el después de su implementación.

Finalmente se hará un estudio comparativo entre los lenguajes Lisp y Scala, detallando su implementación, similitudes y diferencias en base a las funciones de la solución planteada.

Desarrollo

2.1-Fundamentos del diseño funcional adaptado

2.1.1-Definición del paradigma funcional:

La programación funcional es un enfoque donde el programa se construye a partir de funciones matemáticas que reciben datos de entrada y devuelven resultados sin modificar nada fuera de ellas. El objetivo es que cada función sea fácil de entender, probar y reutilizar en distintos contextos.

En este paradigma se evita alterar variables globales, escribir en ficheros de forma descontrolada o depender de estados cambiantes. En lugar de eso, se prioriza el uso de funciones puras, colecciones inmutables y transformaciones encadenadas. **La lógica se centra en qué se quiere calcular y no en cada paso para conseguirlo.**[1]

2.1.2-Principios fundamentales de la programación Funcional

La programación funcional se apoya en varios principios que definen su forma de trabajar. Comprender estos pilares ayuda a tomar mejores decisiones de diseño y a escribir código que aproveche realmente este paradigma.

A continuación, se describen los fundamentos más importantes, que aparecen de manera recurrente en lenguajes como Haskell, Scala, Erlang, Elixir, Python o JavaScript cuando se utiliza de forma funcional.

- **Funciones puras y sin efectos secundarios:** Son la base del modelo y permiten predecir qué ocurre sin revisar todo el programa.
- **Inmutabilidad de datos:** Evita que la información cambie de forma silenciosa y brinda seguridad en entornos concurrentes.
- **Funciones de orden superior:** Otorgan flexibilidad para componer comportamientos a partir de piezas simples.

- **Transparencia referencial:** Posibilita sustituir una expresión por su valor sin alterar el resultado global.
- **Composición de funciones:** Permite encadenar operaciones para construir pipelines de datos claros y concisos.[2]

2.1.3- Principios aplicados al sistema:

El diseño de nuestro sistema de semáforos se basó en los principios del paradigma funcional. Para la resolución de este, se construyó el sistema mediante funciones que reciben parámetros y devuelven un resultado determinado.

Cada una de estas funciones es una función pura, ya que para los mismos parámetros de entrada siempre produce el mismo resultado y no depende ni modifica estados externos. Esto garantiza un comportamiento predecible del sistema.

Además, el modelo evita la modificación directa de datos, ya que, en lugar de cambiar un estado existente del semáforo, se generan nuevos resultados a partir de los valores recibidos como entrada.

2.1.4-Implementación del sistema de semáforos

Para el desarrollo del sistema de semáforos se abordaron los distintos requerimientos establecidos en la consigna. Para la realización de este, cada miembro desarrolló de manera independiente una o más funciones en base a los requerimientos dados.

Una vez finalizada la implementación individual de las funciones, se integró cada una de ellas en un único sistema, verificando su correcto funcionamiento y la compatibilidad entre los mismos. A continuación, se detalla su versión final:

El sistema final implementa el ciclo semafórico con intermitencia de seguridad. El orden utilizado es:

rojo → amarillo-intermitente → verde → amarillo-intermitente →
amarillo → amarillo-intermitente → rojo

Los tiempos utilizados por la versión final son:

- Rojo: 90 segundos.
- Amarillo intermitente: 3 segundos antes de cada cambio.
- Verde: 120 segundos.
- Amarillo: 6 segundos.
- Duración total del ciclo: 225 segundos.

Función transicion:

La función transicion recibe el estado actual del semáforo y el color solicitado. Devuelve una lista con el estado actual y la acción correspondiente si el cambio es válido. En la versión final se incorporó el estado amarillo-intermitente como paso obligatorio entre los cambios principales.

(transicion 'en-rojo' 'amarillo-intermitente')

:: Resultado: (EN-ROJO "cambiar-a-amarillo-intermitente")

Ejemplo de transición inválida:

(transicion 'en-amarillo' 'verde')

:: Resultado: (EN-AMARILLO ACCION-POR-DEFECTO)

Función timer:

La función timer determina el estado del semáforo según el segundo recibido. La versión final utiliza un ciclo de 225 segundos.

0 a 89 → EN-ROJO

90 a 92 → AMARILLO-INTERMITENTE

93 a 212 → EN-VERDE

213 a 215 → AMARILLO-INTERMITENTE

216 a 221 → EN-AMARILLO

222 a 224 → AMARILLO-INTERMITENTE

225 → EN-ROJO

La función valida que el tiempo ingresado sea un número entero mayor o igual a cero.

Función log-auditoria:

La función log-auditoria registra los cambios de estado del semáforo. Utiliza local-time para convertir timestamps Unix en fechas y horas legibles.

[2026-06-10 14:30:00] La luz ha cambiado de EN-ROJO a EN-VERDE

Función duración-ciclo:

La función duracion-ciclo calcula la duración total de un ciclo completo. En la versión final suma rojo, verde, amarillo y tres períodos de intermitencia.

(duracion-ciclo)

:: Resultado: 225

Función recomendación-ciclo:

La función recomendacion-ciclo evalúa si la duración total del ciclo se encuentra dentro de los parámetros establecidos. Si supera el máximo definido, recomienda reducir la duración del ciclo.

(recomendacion-ciclo (duracion-ciclo))

:: Resultado: REDUCIR-DURACION-DEL-CICLO

Función ciclos-por-tiempo:

La función ciclos-por-tiempo recibe una cantidad de minutos, la convierte a segundos y calcula cuántos ciclos completos entran en ese

período. Al utilizar duracion-ciclo, se adapta automáticamente al ciclo final de 225 segundos.

(ciclos-por-tiempo 15)

:: Resultado: 4

Funciones distribucion-una-hora y verificaciones

Estas funciones calculan la distribución temporal real durante una hora completa. En la versión final se consideran los tres períodos de intermitencia dentro de cada ciclo.

(distribucion-una-hora 90 120 6 3)

:: Resultado: (1440 1920 96 144)

(verificaciones 90 120 6)

:: Resultado aproximado:

:: ((ROJO 40.0) (VERDE 53.333336) (AMARILLO 2.6666667)
(AMARILLO-INTERMITENTE 4.0))

Función informe

La función informe genera un archivo de texto con los registros de ejecución del sistema semafórico. Esta función forma parte de la persistencia de datos de la Iteración 2.

(informe

'("2026-06-15 14:30:15 - Transición: ROJO -> AMARILLO-
INTERMITENTE"

"2026-06-15 14:30:18 - Transición: AMARILLO-INTERMITENTE
-> VERDE"))

:: Resultado: INFORME-GENERADO

2.1-Justificación de librerías

2.2.2- Quicklisp y librerías seleccionadas

Quicklisp es el método original para obtener y cargar bibliotecas en un programa Common Lisp. Por defecto tiene una única caché global de paquetes compartida por todos los programas de tu máquina, y extrae paquetes de un repositorio actualizado de forma conservadora también llamado (quizá de forma confusa) Quicklisp. [3]

2.2.3- Elección de librería local-time

Se seleccionó la librería local-time debido a que permite trabajar de manera sencilla con fechas y horas en Common Lisp. Su incorporación tuvo como objetivo mejorar el sistema de auditoría del proyecto, reemplazando la representación del tiempo en formato Unix (Epoch) por una fecha y hora legibles para los usuarios.

La utilización de esta librería facilita la conversión, manipulación y formateo de valores temporales, permitiendo mostrar registros de auditoría en un formato más comprensible, como, por ejemplo: **[2026-05-16 14:30:25]**. De esta manera, se mejora la legibilidad de los eventos registrados y se simplifica la interpretación de la información generada por el sistema.

Además, la librería se integra fácilmente mediante Quicklisp, cuenta con una amplia utilización dentro de la comunidad de Common Lisp y ofrece las funcionalidades necesarias para cumplir con los requerimientos planteados en la consigna sin necesidad de implementar mecanismos adicionales para el manejo de fechas y horas.

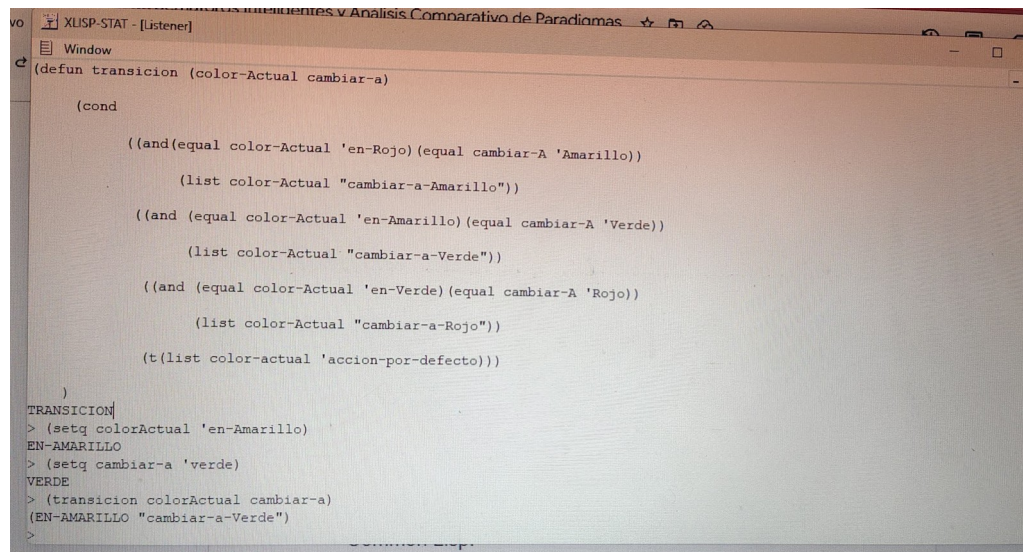
2.3.1-Bitácora de errores y depuración

Durante el desarrollo del sistema de semáforos se nos presentaron diversos inconvenientes que requirieron tareas de análisis y modificación. Entre los principales problemas detectados se

encontraron errores de lógica en algunas funciones, dificultades en la interpretación de ciertos requerimientos de la consigna y la falta de validaciones adicionales. A continuación, se detallan los errores identificados en cada función, junto con las soluciones implementadas para su corrección.

2.3.2-Función Transición:

Durante el desarrollo de la dicha función se presentó un problema en la lógica de transición de colores debido a que en la consigna se especificaba que dicha transición era rojo → amarillo → verde.



```
vo XLIISP-STAT - [listener]
Window
(defun transicion (color-Actual cambiar-a)
  (cond
    ((and(equal color-Actual 'en-Rojo) (equal cambiar-A 'Amarillo))
      (list color-Actual "cambiar-a-Amarillo"))
    ((and (equal color-Actual 'en-Amarillo) (equal cambiar-A 'Verde))
      (list color-Actual "cambiar-a-Verde"))
    ((and (equal color-Actual 'en-Verde) (equal cambiar-A 'Rojo))
      (list color-Actual "cambiar-a-Rojo"))
    (t(list color-actual 'accion-por-defecto)))
  )
TRANSICION
> (setq colorActual 'en-Amarillo)
EN-AMARILLO
> (setq cambiar-a 'verde)
VERDE
> (transicion colorActual cambiar-a)
(EN-AMARILLO "cambiar-a-Verde")
>
```

Sin embargo, al analizar el funcionamiento real de un semáforo, se verificó que la secuencia correcta es **rojo** → **verde** → **amarillo** → **rojo**.

```

XUSP-STAT - [listened]
Window
> (setq cambiar-a 'verde)
VERDE
> (transicion colorActual cambiar-a)
(EN-AMARILLO "cambiar-a-Verde")
> (defun transicion (color-Actual cambiar-a)
  (cond
    ((and(equal color-Actual 'en-Rojo) (equal cambiar-A 'Verde))
      (list color-Actual "cambiar-a-Verde"))
    ((and (equal color-Actual 'en-Verde) (equal cambiar-A 'Amarillo))
      (list color-Actual "cambiar-a-Amarillo"))
    ((and (equal color-Actual 'en-Amarillo) (equal cambiar-A 'Rojo))
      (list color-Actual "cambiar-a-Rojo"))
    (t (list color-Actual 'accion-por-defecto)))
  )
)
TRANSICION
> (setq colorActual 'en-Verde)
EN-VERDE
> (setq cambiar-A 'Amarillo)
AMARILLO
> (transicion colorActual cambiar-A)
(EN-VERDE "cambiar-a-Amarillo")
>
  
```

2.3.3- Función timer:

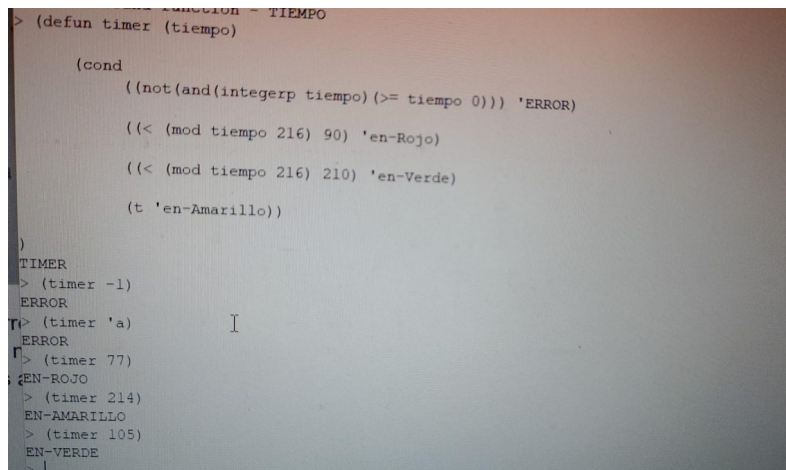
Durante el desarrollo de esta función tuvimos el mismo inconveniente que en la función anterior, ya que primeramente consideramos la secuencia **rojo** → **amarillo** → **verde**, pero posteriormente se determinó que el ciclo correcto debía ser **rojo** → **verde** → **amarillo** → **rojo**. Como consecuencia, fue necesario revisar y ajustar los intervalos de tiempo asociados a cada estado para reflejar correctamente el funcionamiento del semáforo.

```

> (defun timer (tiempo)
  (cond
    ((< (mod tiempo 216) 90) 'en-rojo)
    ((< (mod tiempo 216) 96) 'en-amarillo)
    (t 'en-verde)
  )
)
TIMER
> (timer q)
error: unbound variable - Q
> (timer 93)
EN-AMARILLO
> (timer 213)
EN-VERDE
>
  
```

El segundo error que tuvimos en esta función fue la falta de valoración numérica y que sean valores positivos, como se puede observar en la imagen de arriba.

Para su solución se incorporaron validaciones que permiten comprobar que el tiempo ingresado sea un valor numérico y mayor o igual a cero.



```
> (defun timer (tiempo)
  (cond
    ((not (and (integerp tiempo) (>= tiempo 0))) 'ERROR)
    ((< (mod tiempo 216) 90) 'en-Rojo)
    ((< (mod tiempo 216) 210) 'en-Verde)
    (t 'en-Amarillo))
  )
  )
TIMER
> (timer -1)
ERROR
> (timer 'a)
ERROR
> (timer 77)
EN-ROJO
> (timer 214)
EN-AMARILLO
> (timer 105)
EN-VERDE
```

2.3.4-Función log-auditoría:

Durante el desarrollo de esta función tuvimos un fallo en la estructura local-time:

Al integrar la librería local-time al requerimiento 3 se detectó un problema la función timestring no podía procesar correctamente números enteros grandes al no estar estructurados como hora, minuto y segundo.

Para resolverlo se llegó a la conclusión de agregar la función (unix-to-timestamp) que al anidar dentro del let transforma el entero en una estructura que timestring pueda procesar correctamente.

```

1
2 ;; ERROR: Se pasa el entero 'timestamp' directamente a format-timestring
3 (defun log-auditoria (timestamp color-anterior color-nuevo)
4   (let* ((fecha hora (format-timestring nil timestamp)) ;; <--FALLA: El entero no es un objeto timestamp
5          (format '(:year 4) "-" (:month 2) "-" (:day 2) " "
6                  (:hour 2) ":" (:min 2) ":" (:sec 2))))
7     (format t "~%[~A] La luz ha cambiado de ~A a ~A"
8             fecha hora color-anterior color-nuevo)))
9
10
11 ;; SOLUCIÓN: Se introduce 'unix-to-timestamp' para crear la abstracción correcta
12 (defun log-auditoria (timestamp color-anterior color-nuevo)
13   (let* ((ts-obj (unix-to-timestamp timestamp)) ;; <-- SOLUCION: Conversión a objeto valida
14          (fecha hora (format-timestring nil ts-obj)) ;; Ahora recibe el objeto estructurado
15          (format '(:year 4) "-" (:month 2) "-" (:day 2) " "
16                  (:hour 2) ":" (:min 2) ":" (:sec 2))))
17     (format t "~%[~A] La luz ha cambiado de ~A a ~A"
18             fecha hora
19             color-anterior
20             color-nuevo)))

```

2.3.5-Función duración-ciclo:

Interpretación ambigua del Requerimiento 4

Durante el desarrollo de la función duración-ciclo surgió una dificultad conceptual debido a la redacción del requerimiento.

La consigna indicaba que la función debía recibir “un número determinado de segundos”, pero al mismo tiempo solicitaba calcular la duración total de un ciclo completo del semáforo. Esta redacción resulta ambigua, ya que la duración del ciclo depende de la suma de los tiempos correspondientes a rojo, verde y amarillo. Por lo tanto, no nos quedaba claro si la función debía recibir un único valor de tiempo, si debía trabajar con valores fijos o si debía recibir por separado la duración de cada color.

Para resolver esta ambigüedad, decidimos implementar la función con tres parámetros opcionales mediante &optional:

tiempo de rojo;

tiempo de amarillo;

tiempo de verde.

Cuando no se proporcionan argumentos, la función utiliza automáticamente los valores establecidos por las reglas actuales del sistema: 90 segundos para rojo, 6 segundos para amarillo y 120 segundos para verde.

Esta decisión permitió evitar el ingreso repetido de los mismos parámetros cada vez que se ejecuta la función y, al mismo tiempo, conservar la posibilidad de evaluar configuraciones alternativas sin modificar el código.

De esta manera, la solución no sólo resolvió la ambigüedad del requerimiento, sino que también mejoró la flexibilidad, reutilización y claridad de la función

```
;;  
(defun duracion-ciclo (&optional  
                      (tiempo-rojo 90)  
                      (tiempo-amarillo 6)  
                      (tiempo-verde 120))  
  
  (cond  
    ((or (not (numberp tiempo-rojo))  
         (not (numberp tiempo-amarillo))  
         (not (numberp tiempo-verde)))  
     'error-tiempos-no-numericos)  
  
    ((or (<= tiempo-rojo 0)  
         (<= tiempo-amarillo 0)  
         (<= tiempo-verde 0))  
     'error-tiempos-no-positivos)  
  
    (t  
     (+ tiempo-rojo  
        tiempo-amarillo  
        tiempo-verde))))
```

2.3.6-Función suma-tiempo:

Error en el cálculo de porcentajes sobre un ciclo en lugar de una hora completa

Durante la implementación inicial del Requerimiento 6, la distribución porcentual de los colores se calculó tomando como referencia un único ciclo semafórico de 216 segundos.

La primera versión dividía el tiempo de cada color por la duración total del ciclo:

rojo: 90 segundos;

verde: 120 segundos;

amarillo: 6 segundos.

Este procedimiento permitía obtener correctamente la proporción de cada color dentro de un ciclo, pero no respondía de manera exacta a la consigna, que solicitaba calcular la distribución temporal en un período completo de una hora.

El problema se detectó al comprobar que 3600 segundos no es el múltiplo exacto de 216. En una hora se completan 16 ciclos, equivalentes a 3456 segundos, y quedan 144 segundos adicionales.

Como el período analizado comienza en rojo y el orden del ciclo es rojo, verde y amarillo, el tiempo restante se distribuye de la siguiente manera:

90 segundos adicionales para rojo;

54 segundos adicionales para verde;

0 segundos adicionales para amarillo.

Para corregir el error se incorporó la función distribución-una-hora, encargada de calcular la cantidad de ciclos completos, el tiempo restante y su distribución según el orden de los estados.

De esta forma se obtuvieron los tiempos reales de cada color durante una hora:

rojo: 1530 segundos;

verde: 1974 segundos;

amarillo: 96 segundos.

Posteriormente, la función porcentaje-colores convierte estos valores a porcentajes sobre los 3600 segundos totales.

Los resultados corregidos son aproximadamente:

rojo: 42,5 %;

verde: 54,83 %;

amarillo: 2,67 %.

La corrección permitió diferenciar entre la proporción teórica de un ciclo y la distribución efectiva de los estados durante una hora completa.

2.4.1- Parte teórica Comparativa:

2.4.2- Introducción a lenguaje Scala

Scala es un lenguaje de programación moderno multiparadigmático diseñado para expresar patrones comunes de programación de forma concisa, elegante y segura para tipos. Integra sin problemas características de lenguajes orientados a objetos y funcionales.

Scala es orientado a objetos

Scala es un lenguaje puramente orientado a objetos en el sentido de que todo valor es un objeto. Los tipos y comportamientos de los objetos se describen por clases y rasgos. Las clases pueden ampliarse mediante subclases y utilizando un mecanismo flexible de composición basado en mixina como sustituto limpio de la herencia múltiple.

Scala es funcional

Scala es también un lenguaje funcional en el sentido de que toda función es un valor. Scala proporciona una sintaxis ligera para definir

funciones anónimas, soporta funciones de orden superior, permite anidar funciones y soporta currying. Las clases de casos de Scala y su soporte incorporado para la búsqueda de patrones proporcionan la funcionalidad de los tipos algebraicos, que se utilizan en muchos lenguajes funcionales. Los objetos singleton proporcionan una forma conveniente de agrupar funciones que no son miembros de una clase.

Scala se tipifica estáticamente

El sistema de tipos expresivo de Scala impone, en tiempo de compilación, que las abstracciones se utilicen de manera segura y coherente. En particular, el sistema de tipos soporta:

- Clases genéricas
- Anotaciones de variación
- Límites de tipo superior e inferior
- Clases internas y miembros de tipo abstracto como miembros de objetos
- Tipos compuestos
- Autorreferencias explícitamente tipadas
- Parámetros implícitos y conversiones
- Métodos polimórficos

La inferencia de tipo significa que el usuario no está obligado a anotar código con información redundante de tipo. En combinación, estas características proporcionan una base poderosa para la reutilización segura de abstracciones de programación y para la extensión segura de tipos del software.

Scala es extensible

En la práctica, el desarrollo de aplicaciones específicas de dominio a menudo requiere extensiones de lenguaje específicas de cada

dominio. Scala ofrece una combinación única de mecanismos de lenguaje que facilitan la incorporación de nuevos constructos de lenguaje en forma de librerías. En muchos casos, esto puede hacerse sin utilizar herramientas de metaprogramación como macros.

Por ejemplo:

- Las clases implícitas permiten añadir métodos de extensión a tipos existentes.
- La interpolación de cadenas es extensible por el usuario con interpoladores personalizados.

Scala interopera

Scala está diseñado para interoperar bien con el popular Java Runtime Environment (JRE). En particular, la interacción con el lenguaje de programación Java orientado a objetos es lo más fluida posible. Funciones más recientes de Java como SAM, lambdas, anotaciones y genéricos tienen análogos directos en Scala. Aquellas características de Scala sin análogos de Java, como parámetros por defecto y con nombre, se compilan lo más fielmente posible a Java. Scala tiene el mismo modelo de compilación (compilación separada, carga dinámica de clases) que Java y permite el acceso a miles de bibliotecas de alta calidad existentes.[4]

Industrias y áreas donde se utiliza Scala

Scala es utilizado principalmente en sectores que requieren procesamiento eficiente de grandes cantidades de datos, sistemas distribuidos y aplicaciones de alto rendimiento. Entre las principales industrias se encuentran:

- **Big Data y análisis de datos:** gracias a su integración con herramientas como Apache Spark, una de las plataformas más utilizadas para procesamiento masivo de datos.
- **Tecnología y redes sociales:** para el desarrollo de servicios backend y sistemas distribuidos.
- **Servicios financieros y banca:** debido a su seguridad de tipos y capacidad para manejar aplicaciones complejas y concurrentes.
- **Comercio electrónico:** para sistemas de recomendación, procesamiento de transacciones y análisis de comportamiento de usuarios.
- **Streaming y entretenimiento digital:** en aplicaciones que requieren alta escalabilidad y procesamiento de grandes volúmenes de información.

Empresas que utilizan Scala

Algunas empresas reconocidas que han utilizado Scala en distintos proyectos son:

- Netflix
- LinkedIn
- Twitter
- Airbnb
- The Guardian

2.4.3-Comparación de funciones en lisp y scala

Funciones en lisp:

Sistema de semáforo Inteligente

```
1 (defun transicion (color-Actual cambiar-a)
2
3   (cond
4
5     ((and(equal color-Actual 'en-Rojo)(equal cambiar-A 'Verde))
6
7       (list color-Actual "cambiar-a-Verde"))
8
9     ((and (equal color-Actual 'en-Verde)(equal cambiar-A 'Amarillo))
10
11       (list color-Actual "cambiar-a-Amarillo"))
12
13     ((and (equal color-Actual 'en-Amarillo)(equal cambiar-A 'Rojo))
14
15       (list color-Actual "cambiar-a-Rojo"))
16
17     (t(list color-actual 'accion-por-defecto)))
18
19 )
20
```

```
101> (defun timer (tiempo)
    (cond
      ((not(and(integerp tiempo)(>= tiempo 0))) 'ERROR)
      ((< (mod tiempo 216) 90) 'en-Rojo)
      ((< (mod tiempo 216) 210) 'en-Verde)
      (t 'en-Amarillo))
    )
    TIMER
    > (timer -1)
    ERROR
    > (timer 'a)
    ERROR
    > (timer 77)
    EN-ROJO
    > (timer 214)
    EN-AMARILLO
    > (timer 105)
    EN-VERDE
```

Funciones en Scala:

Sistema de semáforo Inteligente

```
1 object Semaforo {
2
3   def transicion(colorActual: String, cambiarA: String): (String, String) = {
4
5     (colorActual, cambiarA) match {
6
7       case ("en-Rojo", "Verde") =>
8         ("en-Rojo", "cambiar-a-Verde")
9
10      case ("en-Verde", "Amarillo") =>
11        ("en-Verde", "cambiar-a-Amarillo")
12
13      case ("en-Amarillo", "Rojo") =>
14        ("en-Amarillo", "cambiar-a-Rojo")
15
16      case _ =>
17        (colorActual, "accion-por-defecto")
18    }
19  }
20
21
22  def timer(tiempo: Int): String = {
23    if (tiempo < 0)
24      "Error"
25    else {
26      val ciclo = tiempo % 216
27
28      if (ciclo < 90) "en-Rojo"
29      else if (ciclo < 210) "en-Verde"
30      else "en-Amarillo"
31    }
32  }
33
34  println(Semaforo.transicion("en-Rojo", "Verde")) (): scala.Unit
35  println(Semaforo.transicion("en-Verde", "Amarillo")) (): scala.Unit
36  println(Semaforo.transicion("en-Amarillo", "Rojo")) (): scala.Unit
37  println(Semaforo.transicion("celeste", "Amarillo")) (): scala.Unit
38
39  println(Semaforo.timer(36)) (): scala.Unit
40  println(Semaforo.timer(97)) (): scala.Unit
41  println(Semaforo.timer(188)) (): scala.Unit
42  println(Semaforo.timer(211)) (): scala.Unit
43  println(Semaforo.timer(366)) (): scala.Unit
44  println(Semaforo.timer(574)) (): scala.Unit
```


Sistema de semáforo Inteligente

Aspecto	Common Lisp	Scala
Definición de función	Se utiliza defun.	Se utiliza def.
Función transición	(defun transicion (estado) ...)	def transicion(estado: String): String = ...
Función timer	(defun timer (tiempo) ...)	def timer(tiempo: Int): String = ...
Parámetros	No requieren declaración.	Requieren tipos explícitos.
Tipo de retorno	Determinado dinámicamente.	Declarable explícitamente.
Condicionales	cond e if.	if, else if y else.
Tipado	Dinámico.	Estático.
Sintaxis	Basada en listas/paréntesis.	Similar a Java.
Detección errores	Tiempo de ejecución.	Tiempo de compilación.
Paradigma funcional	Funciones puras/recursión.	Funciones puras/inmutabilidad.
Implementación transición	Utiliza cond y comparación mediante equal.	Utiliza if/else if y operadores ==.
Implementación timer	Se basa en condiciones sobre intervalos.	Utiliza condiciones, validaciones y cálculo del ciclo.

2.4.4- Estructura del semáforo en Scala:

Para la implementación del semáforo en Scala se utilizó un Singleton Object (**object**). La decisión se tomó porque las funciones **transición** y **timer** son funciones puras que no necesitan almacenar estado interno ni crear múltiples instancias de un semáforo.

Desde el diseño funcional, un **object** resulta adecuado porque permite agrupar funciones relacionadas en una única unidad lógica sin necesidad de utilizar variables mutables. Cada resultado depende exclusivamente de los parámetros recibidos, manteniendo el principio de transparencia referencial propio de la programación funcional.

No se utilizó una clase tradicional porque no era necesario crear objetos mediante **new**, ni almacenar atributos propios para cada semáforo. Tampoco se utilizaron **case class**, ya que estas son más

apropiadas para modelar estructuras de datos inmutables y representar entidades con atributos, mientras que en este caso solo se requería implementar operaciones sobre datos de entrada.

Por estas razones, el uso de un **Singleton Object** permitió una solución simple, funcional y coherente con los requerimientos del ejercicio.

2.4.5- Comparación entre listas en Scala y Common Lisp

Scala y Common Lisp permiten trabajar con listas mediante funciones de orden superior, aunque presentan diferencias sintácticas importantes.

En Scala, las operaciones sobre listas suelen expresarse mediante métodos como **map**, **filter** y **foreach**, aplicados directamente sobre la colección. Por ejemplo:

```
lista.filter(x => x > 0)
```

La sintaxis resulta cercana a otros lenguajes modernos y suele ser más fácil de leer para programadores acostumbrados a la programación orientada a objetos.

En Common Lisp, las operaciones equivalentes suelen expresarse mediante funciones como **mapcar**, **remove-if** o funciones recursivas definidas por el programador. Por ejemplo:

```
(remove-if #'minusp lista)
```

Aunque Common Lisp ofrece gran flexibilidad y potencia, su notación prefija y el uso intensivo de paréntesis pueden dificultar la lectura inicial para quienes no están familiarizados con el lenguaje.

Consideramos que Scala presenta una mayor legibilidad para este tipo de operaciones debido a su sintaxis más cercana al lenguaje natural y a que los métodos se encadenan directamente sobre las listas. Sin

embargo, Common Lisp ofrece una gran expresividad funcional y un control más explícito sobre la manipulación de datos

3-Conclusion

En conclusión, el desarrollo de este Trabajo Práctico Integrador nos permitió poner en práctica todos los conocimientos adquiridos durante la cursada. A través del paradigma funcional, pudimos comprender cómo es posible resolver problemas del mundo real mediante funciones puras, inmutabilidad y una lógica basada en la transformación de datos.

Asimismo, la implementación en un nuevo lenguaje de programación representó un desafío importante, ya que nos obligó a investigar su sintaxis, comprender sus características y adaptar la lógica desarrollada originalmente en Common Lisp. Esta experiencia nos permitió observar que, a pesar de las diferencias sintácticas, los principios fundamentales de la programación funcional se mantienen y pueden aplicarse de manera similar en distintos lenguajes, como Scala.

Este trabajo no solo nos desafió como grupo, sino también como futuros profesionales, ya que durante su desarrollo enfrentamos diversos problemas técnicos y conceptuales que logramos resolver mediante el trabajo colaborativo, la investigación y el análisis. Como resultado, nos llevamos una experiencia enriquecedora, nuevos conocimientos y una mejor comprensión tanto del paradigma funcional como de la importancia del trabajo en equipo en el desarrollo de soluciones.

Referencia bibliografía:

- [1][Programación Funcional: Qué es, Lenguaje y Características](#)
- [2][Programación Funcional: Qué es, Lenguaje y Características](#)
- [3][Ceceo común - ArchWiki](#)
- [4][Introducción | Tour por Scala | Documentación de Scala](#)